HW8 for CS 6210

You may (and probably should) talk about problems with the each other, with the TA, and with me, providing attribution for any good ideas you might get. Your final write-up should be your own.

```
using LinearAlgebra
```

```
using Plots
```

```
using FFTW
```

1. Just Fourier Fun

Let $T_n \in \mathbb{R}^{n \times n}$ be the standard tridiagonal with 2 on the main diagonal and -1 on the off-diagonals. This matrix has the eigenvalue decomposition

$$T_n = V \Lambda V^*$$

where the eigenvector matrix $V = V^*$ is the (type-I) discrete sine transform matrix with entries

$$v_{jk} = \sqrt{\frac{2}{n+1}} \sin\left(\frac{jk\pi}{n+1}\right)$$

and the associated eigenvalues are

$$\lambda_k = 2\left(1 - \cos\left(\frac{k\pi}{n+1}\right)\right).$$

The matrix V can be multiplied by a vector in $O(n \log n)$ time using the fast Fourier transform (FFT) algorithm. We provide fast routines (both called `dst`) for the maps $x \mapsto Vx$ and $A \mapsto V^*AV$ below.

```
dst (generic function with 2 methods)
```

```
begin
    T1D(n) = SymTridiagonal(2*ones(n), -ones(n-1))
    λ1D(n) = [2*(1-cos(k*π/(n+1))) for k = 1:n]
    dst(v :: Vector{Float64}) = FFTW.r2r(v, FFTW.RODFT00)/sqrt((length(v)+1)*2)
    dst(A :: Matrix{Float64}) = FFTW.r2r(A, FFTW.RODFT00)/sqrt(prod(size(A).+1))/2
end
```

As a sanity check, let's make sure that the 2D DST ($A \mapsto V^*AV$) diagonalizes T with the correct eigenvalues.

```
1.7726889865138102e-16
```

```
norm(dst(Matrix(T1D(10))) - Diagonal(λ1D(10))) / norm(λ1D(10))
```

We can use the eigendecomposition $T = V\Lambda V^*$ to solve the equation

$$TU + UT = -h^2 F$$

by pre- and post-multiplying the equation by V^* and V to get

$$(V^*TV)(V^*UV) + (V^*UV)(V^*TV) = -h^2 V^*FV$$

or

$$\Lambda\tilde{U} + \tilde{U}\Lambda = -h^2\tilde{F}$$

where $\tilde{U} = V^*UV$ and $\tilde{F} = V^*FV$. This gives the scalar equations

$$u_{jk} = -h^2 f_{jk}/(\lambda_j + \lambda_k)$$

and we can recover U by undoing the transformation.

Using this algorithm and intelligently applying the `dst` codes, rewrite the following $O(n^6)$ solver for the n-by-n discrete 2D Poisson problem to run in $O(n^2 \log n)$ time.

```
grid_sample (generic function with 1 method)
```

```
grid_sample(n, f :: Function) = [f(i/(n+1), j/(n+1)) for i=1:n, j=1:n]
```

```
poisson_naive (generic function with 1 method)
```

```
function poisson_naive(n, f :: Function)
    h = 1.0/(n+1)
    F = grid_sample(n, f)
    U = zeros(n, n)
    T = T1D(n)
    U[:] = (kron(T, Matrix(I,n,n)) + kron(Matrix(I,n,n),T))\(-h^2*F[:])
    U
end
```

For your convenience, we also provide a test function.

```
poisson_test (generic function with 2 methods)
```

```
function poisson_test(n, f :: Function, solver :: Function = poisson_naive)
    h = 1.0/(n+1)
    F = grid_sample(n, f)
    U = solver(n, f)
    T = T1D(n)
    R = T*U + U*T + h^2*F
    norm(R)/norm(F)
end
```

```
2.1442922737103646e-17
```

```
poisson_test(10, (x,y) -> x^2 + y^2)
```

```
poisson_dst (generic function with 1 method)
```

```
function poisson_dst(n, f)
    F = grid_sample(n, f)
    h = 1.0 / (n + 1)
    h2 = h * h
    lambda = λ1D(n)
    tilde_F = dst(F)
    tilde_U = similar(tilde_F)
    for j = 1:n, k = 1:n
        tilde_U[j, k] = -h2 * tilde_F[j, k] / (lambda[j] + lambda[k])
    end
    U = dst(tilde_U)

    U
end
```

```
3.785179049028455e-17
```

```
poisson_test(3000, (x,y) -> x^2 + y^2, poisson_dst)
```

From the power method and subspace iteration, T has eigendecomposition $T = V\Lambda V^T$, with V orthogonal/symmetric ($V^T = V$), entries $v_{jk} = \sqrt{2/(n+1)}\sin(jk\pi/(n+1))$, and $\lambda_k = 2(1-\cos(k\pi/(n+1)))$. The 2D operator is $I \otimes T + T \otimes I$, eigendecomposed as $(V \otimes V)(\, I \otimes \Lambda + \Lambda \otimes I\,)(V^T \otimes V^T)$. To solve: Sample $F = \mathbf{grid_sample}(n, f)$. Transform $\tilde{F} = VFV = \mathbf{dst(F)}$. Solve elementwise $\tilde{U}_{jk} = -h^2\tilde{F}_{jk}/(\lambda_j + \lambda_k)$. Inverse transform $U = V\tilde{U}V = \mathbf{dst}(\tilde{U})$. This leverages fast DST via FFT, avoiding large matrices.

Run-time Analysis:

Sampling F: $O(n^2)$ (grid evaluation).

Computing λ: $O(n)$.

DST for $\tilde{F}$: $O(n^2 \log n)$ (2D FFT-based).

Elementwise $\tilde{U}$: $O(n^2)$.

DST for U: $O(n^2 \log n)$.

Total: $O(n^2 \log n)$, scalable (e.g., n=1000 is feasible, vs. naive $O(10^{12})$).

2. A Problem with Potential

Now consider the problem

$$TU + UT + h^2 W \odot U = -h^2 F$$

where $W \odot U$ refers to the elementwise product of W and U. This corresponds to solving the PDE

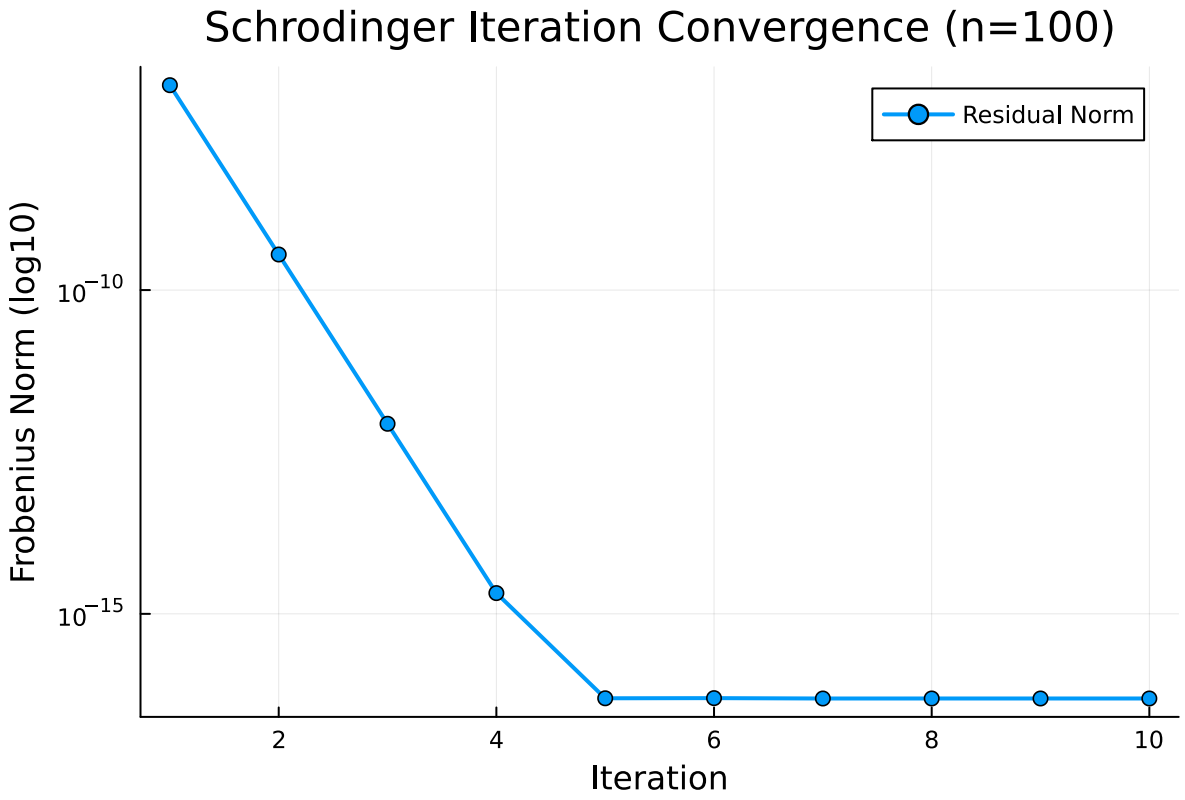
$$(-\nabla^2 + w(x,y))u(x,y) = f(x,y), \quad (x,y) \in (0,1)^2$$

with the homogeneous Dirichlet condition $u = 0$ on the boundary of the domain.

Using one of the solvers above, implement a stationary iteration for this problem where M corresponds to the map $U \mapsto TU + UT$. Try this with the potential $w(x, y) = xy(1 - x)(1 - y)$ (sampled to get W), and give a semi-log plot of the residual norm versus iteration count for ten steps. You should start with the zero matrix as an initial guess. I recommend using a 100-by-100 grid, but you may choose to do this with a 10-by-10 grid if you are having trouble with problem 1.

```
schrodinger_iter (generic function with 1 method)
```

```
function schrodinger_iter(W, F, U, nsteps)
    n = size(F, 1)
    h = 1.0 / (n + 1)
    T = T1D(n)
    lambda = λ1D(n)
    resids = Float64[]
    for step = 1:nsteps
        G = F + W .* U
        tilde_G = dst(G)
        tilde_U = similar(tilde_G)
        for j = 1:n, k = 1:n
            tilde_U[j, k] = -h^2 * tilde_G[j, k] / (lambda[j] + lambda[k])
        end
        U = dst(tilde_U)
        R = T * U + U * T + h^2 * (W .* U + F)
        push!(resids, norm(R) / norm(F))
    end
    U, resids
end
```




```
begin
    w(x,y) = x*y*(1-x)*(1-y)
    f(x,y) = x^2 + y^2
    n = 100
    W = grid_sample(n, w)
    F = grid_sample(n, f)
    U = zeros(n, n)
    U_final, resids = schrodinger_iter(W, F, U, 10)
    plot(1:10, resids, yscale=:log10, marker=:circle, linewidth=2, label="Residual
    Norm", xlabel="Iteration", ylabel="Frobenius Norm (log10)", title="Schrodinger
    Iteration Convergence (n=100)", grid=true, legend=:topright)
end
```

The equation is $TU + UT + h^2 W \odot U = -h^2 F$, rearranged as $TU + UT = -h^2(F + W \odot U)$. This suggests a stationary (Jacobi-like) iteration: solve the Poisson problem with modified right-hand side $U^{k+1} = TU + UT = -h^2(F + W \odot U^k)$. Implement the DST-based solver directly in the loop to handle matrix $G = F + W .* U$. The residual $R = TU + UT + h^2(W \odot U + F)$, normalized by $\|F\|$. This preconditions the variable-coefficient problem with the constant-coefficient Laplacian solver.

```
md"""
The equation is $T U + U T + h^2 W \odot U = -h^2 F$, rearranged as $T U + U T = -h^2
(F + W \odot U)$. This suggests a stationary (Jacobi-like) iteration: solve the
Poisson problem with modified right-hand side $U^{k+1} = T U + U T = -h^2 (F + W
\odot U^k)$. Implement the DST-based solver directly in the loop to handle matrix $G
= F + W .* U$. The residual $R = T U + U T + h^2 (W \odot U + F)$, normalized by
$\|F\|$. This preconditions the variable-coefficient problem with the constant-
coefficient Laplacian solver.

"""
```